

Computer Vision – Assignment 5

Vasiliki Zerva

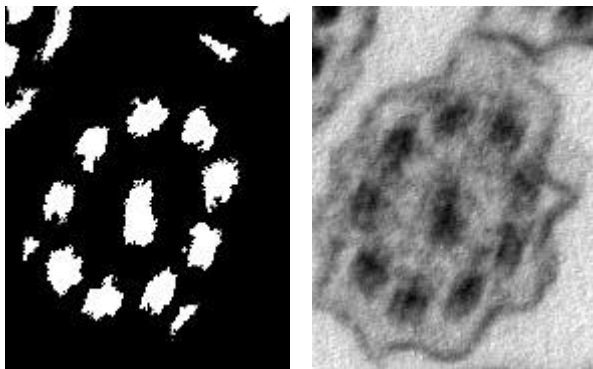
*Faculty of Informatics and Information Technology – Slovak
University of Technology in Bratislava (FIIT – STUBA)*

Morphological Operations

Original Image

```
14 #Show Original Image
15 print("Original Image:")
16 cv2_imshow(img)
```

As an original image, there have been used a binary and a grayscale image :



Structuring Elements

```
17
18 #Structuring Elements
19 kernel_rect = cv.getStructuringElement(cv.MORPH_RECT, (5, 5))
20 kernel_ellipse = cv.getStructuringElement(cv.MORPH_ELLIPSE, (5, 5))
21 kernel_cross = cv.getStructuringElement(cv.MORPH_CROSS, (5, 5))
```

In this code, there being used 3 different Structuring Elements : Rectangle, Ellipse and Cross.

Rectangle kernel

It creates a full 5x5 block of ones. It is the most aggressive in morphology, because it treats all directions equally. It is suitable for cleaning and filling. The kernel mask looks like that :

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

Ellipse kernel :

It is a 5x5 block that creates a circle. It is softer than the rectangle kernel. Suitable for rounder features and natural, organic shapes like cells or tubules. The kernel mask looks like that :

0 0 1 0 0

0 1 1 1 0

1 1 1 1 1

0 1 1 1 0

0 0 1 0 0

Cross Kernel

It focuses more on vertical and horizontal directions. It is less aggressive. The kernel mask looks like that :

0 0 1 0 0

0 0 1 0 0

1 1 1 1 1

0 0 1 0 0

0 0 1 0 0

Change of size and shape

If we change the shape of the kernel in each morphological operation, we will not get the best result. From the result that we will get, the operation of each function will not be clear enough. The shapes that we have chosen for each operation are the most suitable for each function.

For all the structuring elements we have used the size (5,5), which gives a medium effect. If we use a smaller size, for example (3,3), we can still see some results, but they are not that noticeable. That means that the effect of the kernel will exist, but it will not be that intense. However, if we use a bigger size, for example (7,7), there are still going to be results from the kernel, but now they will be more noticeable and more intense. That means that the kernel will have a higher and stronger effect. It is more preferable to use the medium size (5,5).

Erosion and Dilation

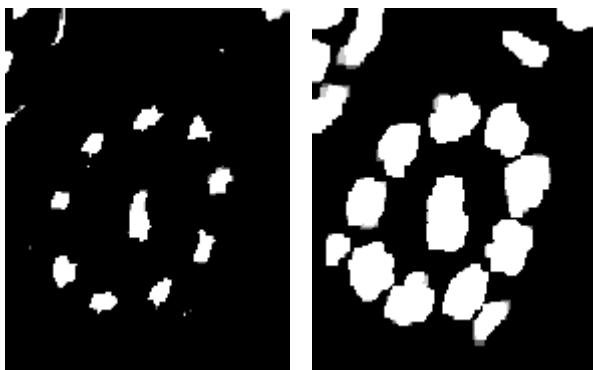
```
23 #Erosion and Dilation
24 erosion = cv.erode(img, kernel_rect)
25 dilation = cv.dilate(img, kernel_rect)
26 print("Erosion:")
27 cv2_imshow(erosion)
28 print("Dilation:")
29 cv2_imshow(dilation)
```

Erosion : it erodes away the boundaries of foreground object. A pixel in the original image (either 1 or 0) will be considered 1 only if all the pixels under the kernel is 1, otherwise it is eroded (made to zero). So the thickness or size of the foreground object decreases or simply white region decreases in the image. It is

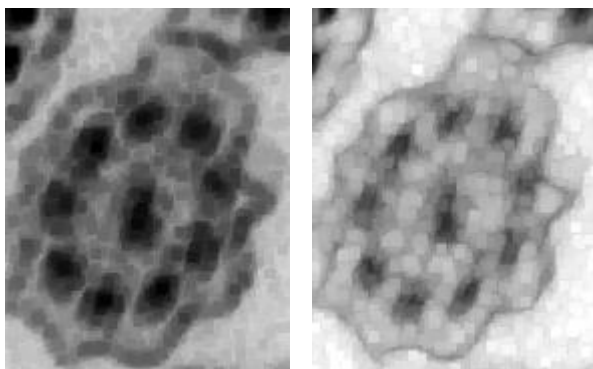
useful for removing small white noises and to detach two connected objects.

Dilation : a pixel element is '1' if at least one pixel under the kernel is '1'. So it increases the white region in the image or size of foreground object increases. In cases like noise removal, erosion is followed by dilation. Because, erosion removes white noises, but it also shrinks our object. So we dilate it. Since noise is gone, they won't come back, but our object area increases. It is also useful in joining broken parts of an object.

In Erosion and Dilation we have used rectangle kernel, so we can observe easier and simpler the differences that they make. For the binary image the results are :



And for the grayscale image the results are :



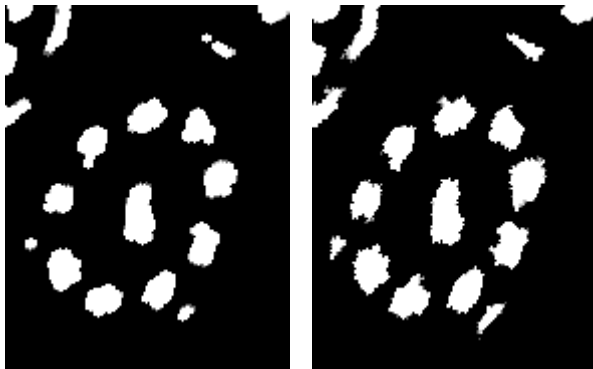
Opening and Closing

```
30
31 #Opening and Closing
32 opening = cv.morphologyEx(img, cv.MORPH_OPEN, kernel_ellipse)
33 closing = cv.morphologyEx(img, cv.MORPH_CLOSE, kernel_cross)
34 print("Opening:")
35 cv2_imshow(opening)
36 print("Closing:")
37 cv2_imshow(closing)
```

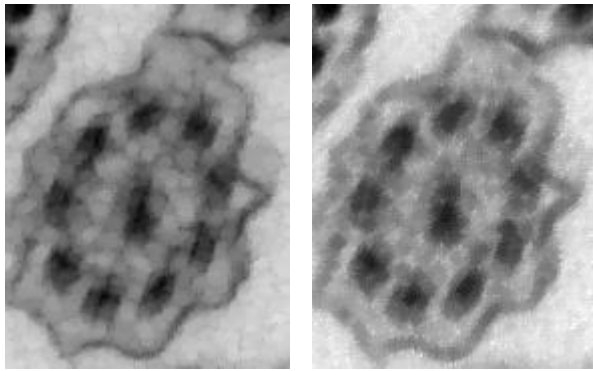
Opening : it is erosion followed by dilation. It is suitable for removing noise.

Closing : it is the reverse of opening : dilation followed by erosion. It is suitable for closing small holes inside the foreground objects or small black points on the object.

There is used Ellipse kernel for Opening, as there is need for a softer removal without damaging the round shapes, and Cross kernel for Closing, as there is need for filling small holes and gaps. For the binary image the results are :



And for the grayscale image the results are :



Gradient, Tophat, Blackhat

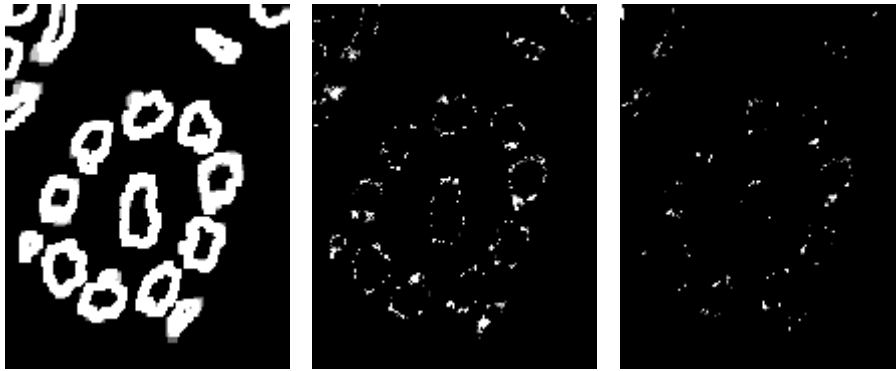
```
39 #Gradient, Tophat, Blackhat
40 gradient = cv.morphologyEx(img, cv.MORPH_GRADIENT, kernel_rect)
41 tophat = cv.morphologyEx(img, cv.MORPH_TOPHAT, kernel_ellipse)
42 blackhat = cv.morphologyEx(img, cv.MORPH_BLACKHAT, kernel_cross)
43 print("Gradient:")
44 cv2_imshow.gradient)
45 print("Tophat:")
46 cv2_imshow(tophat)
47 print("Blackhat:")
48 cv2_imshow(blackhat)
```

Gradient : it is the difference between dilation and erosion of an image.

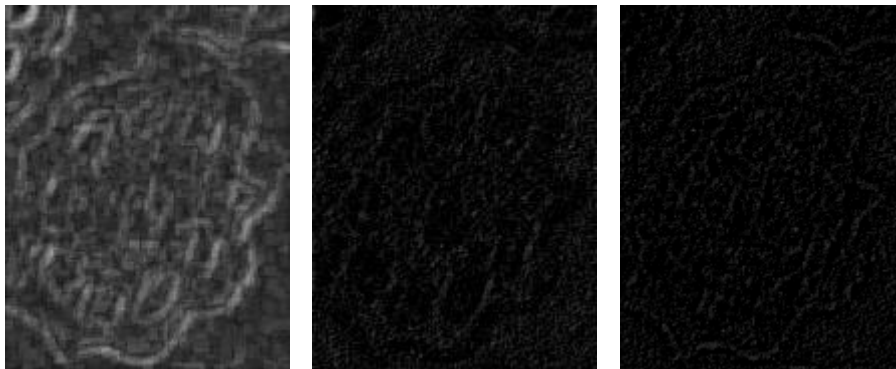
Tophat : it is the difference between the Original image and the Opening of the image.

Blackhat : it is the difference between the Closing of the image and the Original image.

For the binary image the results are :



And for the grayscale image the results are :



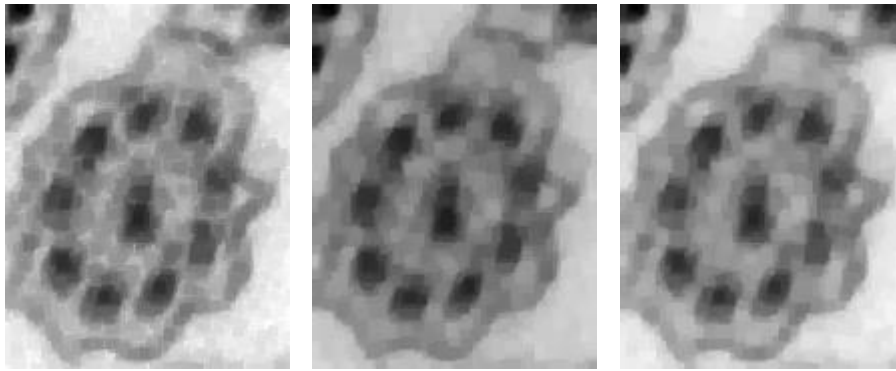
Sequences

```
50 #Sequences: Dilation -> Erosion, Opening -> Closing, Closing -> Opening
51 dilate_erode = cv.erode(cv.dilate(img, kernel_rect), kernel_rect)
52 open_close = cv.morphologyEx(cv.morphologyEx(img, cv.MORPH_OPEN, kernel_rect), cv.MORPH_CLOSE, kernel_rect)
53 close_open = cv.morphologyEx(cv.morphologyEx(img, cv.MORPH_CLOSE, kernel_rect), cv.MORPH_OPEN, kernel_rect)
54 print("Dilation -> Erosion:")
55 cv2.imshow(dilate_erode)
56 print("Opening -> Closing:")
57 cv2.imshow(open_close)
58 print("Closing -> Opening:")
59 cv2.imshow(close_open)
60
```

For the binary image the results are :



And for the grayscale image the results :



Calculation of distance transform

```

61 #Distance Transform
62 if len(img.shape) == 2 :
63     _, binary = cv.threshold(img, 127, 255, cv.THRESH_BINARY)
64     dist_transform = cv.distanceTransform(binary, cv.DIST_L2, 5)
65     dist_display = cv.normalize(dist_transform, None, 0, 255, cv.NORM_MINMAX).astype(np.uint8)
66     cv2.imshow(dist_display)
67 else:
68     print("Distance Transform skipped: Not a binary image (0 and 255).")

```

For the binary image the result is :



And for the grayscale image the result is :

Distance Transform skipped: Not a binary image (0 and 255).

Contour Analysis

```

70 #Make sure it is a binary image
71 #Make sure that it is a binary image
72 _, binary = cv.threshold(img, 127, 255, cv.THRESH_BINARY)
73
74 #Find contours with hierarchy
75 contours, hierarchy = cv.findContours(binary, cv.RETR_CCOMP, cv.CHAIN_APPROX_SIMPLE)
76
77 #Count tubules inside each cilie
78 tubules_per_cilie = {}
79 for i, h in enumerate(hierarchy[0]):
80     parent = h[3] #h[3] is the index of the parent contour
81     if parent != -1: #It's a tubule (inside another contour)
82         tubules_per_cilie[parent] = tubules_per_cilie.get(parent, 0) + 1
83
84 #Results
85 print("Tubules in each cilie:")
86 for cilie_id, count in tubules_per_cilie.items():
87     print(f"  Cilie {cilie_id}: {count} tubule(s)")
88
89 #Draw contours
90 output = cv.cvtColor(img, cv.COLOR_GRAY2BGR)
91 cv.drawContours(output, contours, -1, (0,255,0), 1)
92 cv2.imshow(output)

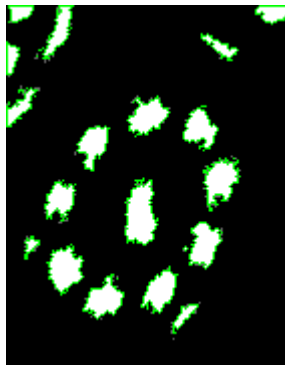
```

Results

```

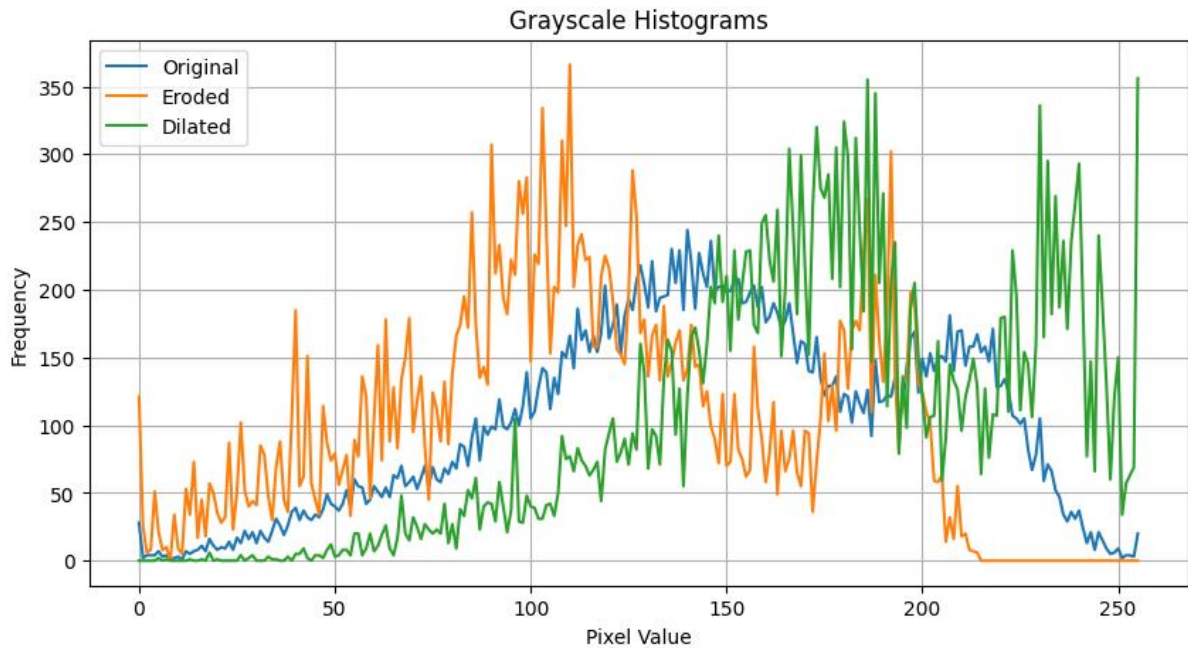
Tubules in each cilie:
  Cilie 2: 3 tubule(s)
  Cilie 8: 2 tubule(s)
  Cilie 12: 1 tubule(s)
  Cilie 14: 1 tubule(s)
  Cilie 16: 2 tubule(s)
  Cilie 20: 3 tubule(s)
  Cilie 27: 1 tubule(s)
  Cilie 31: 1 tubule(s)
  Cilie 33: 1 tubule(s)
  Cilie 35: 3 tubule(s)

```



Histogram Calculation

Results



From the histogram, it is quite clear that for a dilated image the higher the pixel value gets, the more frequency it has, except for the values of 190-220 approximately when the frequency drops. About the eroded image, it can get a high frequency around 80-130 and 180-190 pixels. In all the other pixel values, it has a low frequency and around 210-250 pixels it has 0 frequency.